Thread Synchronization using Condition Variables

Overview

This program demonstrates thread synchronization using POSIX threads (pthread) and condition variables. A simple Producer-Consumer system is implemented where a producer thread generates items and a consumer thread consumes them from a shared buffer.

The shared resource is represented by a counter (buffer_count) that tracks the number of items currently present in the buffer.

Synchronization Mechanisms Used

The program uses the following synchronization primitives:

Mutex

pthread_mutex_t mutex;

The mutex protects the shared resource (buffer_count) and ensures that only one thread can access or modify it at a time.

Condition Variables

pthread_cond_t not_full;
pthread_cond_t not_empty;

Two condition variables are used:

- **not_full** – Signals that the buffer has available space.
- **not_empty** – Signals that the buffer contains at least one item.

These condition variables allow threads to wait efficiently without continuously checking the buffer state.

Producer Behavior

The producer thread generates items and adds them to the buffer.

Before producing an item, it checks whether the buffer is full:

while(buffer_count == BUFFER_SIZE)

If the buffer is full:

- The producer cannot add more items.
- It waits using:

pthread_cond_wait(&not_full, &mutex);

When space becomes available, the producer wakes up and continues execution.

After producing an item, the producer signals:

pthread_cond_signal(&not_empty);

to notify the consumer that an item is available.

Consumer Behavior

The consumer thread removes items from the buffer.

Before consuming an item, it checks whether the buffer is empty:

while(buffer_count == 0)

If the buffer is empty:

- The consumer cannot consume any item.
- It waits using:

pthread_cond_wait(&not_empty, &mutex);

When an item becomes available, the consumer wakes up and continues execution.

After consuming an item, the consumer signals:

pthread_cond_signal(&not_full);

to notify the producer that space is available.

Thread Communication

The producer and consumer communicate through condition variables.

Communication occurs as follows:

1. Producer adds an item and signals not_empty.
2. Consumer wakes up and consumes an item.

3. Consumer signals not_full.
4. Producer wakes up and continues producing.

This signaling mechanism ensures proper coordination between threads without unnecessary CPU usage.

How Synchronization Prevents Inconsistent Behavior

Without synchronization:

- Multiple threads may access shared data simultaneously.
- Buffer values may become incorrect.
- Threads may attempt invalid operations such as consuming from an empty buffer or producing into a full buffer.

With synchronization:

- The mutex ensures safe access to the shared buffer.
- Condition variables ensure that threads wait when operations cannot be performed safely.
- Threads continue execution only when the required condition becomes true.

As a result:

- Data consistency is maintained.
- Invalid operations are prevented.
- Thread execution remains predictable and correct.

Conclusion

This implementation demonstrates safe thread synchronization using POSIX condition variables and mutexes. The producer and consumer threads coordinate efficiently through signaling and waiting mechanisms, ensuring correct access to the shared resource and preventing inconsistent behavior in a concurrent environment.